

Esse arquivo docker-compose.yml é uma "receita" que diz ao Docker: "monte esses três containers, configurados assim, e conecte eles entre si desse jeito".

version: '3.8'

Indica a **versão da sintaxe** do docker-compose sendo usada. É só uma formalidade para o Docker saber quais recursos estão disponíveis nesse arquivo.

services:

A partir daqui, listamos os **containers** que vamos criar. Você tem três: web, db e phpmyadmin. Cada um é um "computador virtual" separado, rodando uma coisa diferente.

Serviço web

yaml

build: .

Ao invés de usar uma imagem pronta, esse container é **construído a partir do Dockerfile** que está na mesma pasta (o . significa "pasta atual") — aquele arquivo que expliquei antes (PHP + Apache).

yaml

container_name: aulaphp_web

Dá um **nome fixo** ao container, pra facilitar identificar ele nos comandos (docker logs aulaphp_web, por exemplo). Sem isso, o Docker geraria um nome aleatório.

yaml

restart: always

Se o container cair ou o computador reiniciar, o Docker **reinicia ele automaticamente**.

yaml

ports:

- "8080:80"

Conecta a **porta 8080 do seu computador** com a **porta 80 dentro do container** (onde o Apache escuta). É por isso que você acessa localhost:8080 no navegador.

yaml

volumes:

- ./src:/var/www/html

"Espelha" a pasta `./src` do seu computador dentro do container, na pasta `/var/www/html`. Assim, quando você edita um arquivo `.php` no seu editor, a mudança aparece instantaneamente dentro do container — sem precisar reconstruir nada.

yaml

depends_on:

- db

Diz ao Docker para **iniciar o container db antes** do web. (Atenção: isso só espera o container *começar* a subir, não garante que o MySQL já esteja pronto para aceitar conexões — foi por isso que tivemos aquele erro de conexão lá atrás.)

yaml

networks:

- crud-network

Coloca esse container dentro da rede interna `crud-network` (explicada mais abaixo), permitindo que ele converse com os outros containers pelo nome.

Serviço db

yaml

image: mysql:8.0

Diferente do web, esse container **não precisa ser construído** — ele usa a imagem pronta oficial do MySQL versão 8.0, baixada direto do Docker Hub.

yaml

environment:

MYSQL_ALLOW_EMPTY_PASSWORD: "yes"

MYSQL_DATABASE: aulaphp

MYSQL_USER: aluno_user

MYSQL_PASSWORD: aluno_pass

Essas são **variáveis de ambiente** que a imagem do MySQL entende automaticamente na primeira vez que sobe:

- Permite senha vazia para o usuário root
- Cria um banco de dados chamado aulaphp
- Cria um usuário `aluno_user` com senha `aluno_pass`, com acesso a esse banco

yaml

ports:

- "3306:3306"

Conecta a porta 3306 do seu computador à porta 3306 do container (a porta padrão do MySQL). Isso permite, por exemplo, conectar nesse banco usando um programa como o DBeaver direto do seu computador.

yaml

volumes:

- db_data:/var/lib/mysql

Aqui é onde o MySQL **guarda os dados de verdade** (as tabelas, os registros). Usamos um "volume nomeado" (db_data) em vez de uma pasta comum, para que os dados **sobrevivam** mesmo se você apagar e recriar o container. Foi exatamente esse volume que tivemos que apagar (down -v) quando o banco ficou "bagunçado" nos testes anteriores.

yaml

- ./db/init.sql:/docker-entrypoint-initdb.d/init.sql

Essa linha copia seu arquivo init.sql para dentro de uma pasta especial que o MySQL **executa automaticamente**, mas **só na primeira vez** que o volume db_data é criado do zero. Foi esse arquivo que criou a tabela cliente.

Serviço phpmyadmin

yaml

image: phpmyadmin/phpmyadmin

Usa a imagem pronta do phpMyAdmin — uma interface visual (em vez de linha de comando) para gerenciar o banco de dados pelo navegador.

yaml

ports:

- "8081:80"

Você acessa em localhost:8081.

yaml

environment:

PMA_HOST: db

PMA_USER: root

PMA_PASSWORD: ""

Diz ao phpMyAdmin **onde e com qual usuário se conectar** ao banco — repare que aqui também usamos db (o nome do serviço), pelo mesmo motivo que expliquei no diagrama: containers se enxergam pelo nome do serviço, não por localhost.

networks: (no final do arquivo)

yaml

networks:

crud-network:

driver: bridge

Aqui é onde a rede crud-network é **definida** (criada). O driver: bridge é o tipo de rede padrão do Docker — basicamente cria uma "rede local privada" só para esses containers conversarem entre si.

volumes: (no final do arquivo)

yaml

volumes:

db_data:

Aqui é onde o volume db_data (usado lá no serviço db) é **declarado**. Ele existe fora dos containers, gerenciado pelo próprio Docker, e persiste os dados do banco mesmo se os containers forem removidos.

Resumindo: esse arquivo cria três containers conectados pela mesma rede interna. O web serve suas páginas PHP, o db guarda os dados em um volume persistente, e o phpmyadmin te dá uma interface visual para olhar esse banco — tudo isso "combinando" com o que já vimos no diagrama anterior.